
Automated ICD-9 Coding on MIMIC-III

Entity-Aware Hierarchical Explainable Coding
with NER Filtering, Transformer Backbone,
and Mullenbach Baseline Replication

Dataset	MIMIC-III · Top-50 ICD-9 Codes
Institution	NIT Calicut, Bioinformatics — Group 9
Baseline	Mullenbach et al., NAACL 2018
GitHub Repo	Automated ICD-9 Coding on MIMIC III

Alina Mary Sam	alina_m250234cs@nitc.ac.in
Ashna Hussain	ashna_m250279cs@nitc.ac.in
Shreeya Kale	shreeya_m250504cs@nitc.ac.in
Swaroop Haridas	swaroop_m250538cs@nitc.ac.in

*This document provides full technical documentation of the implementation
of NER filter methodology pipeline:
architecture, training procedure, evaluation methodology, and explainability
framework.*

Contents

1	Overview & Project Summary	3
1.1	Key Design Decisions	3
1.2	Version History & Bug Fixes (v1 → v3)	3
2	Repository Structure	4
3	Environment & Installation	4
3.1	Hardware Requirements	4
3.2	Dependencies	5
3.3	Quick Setup	5
4	Data Pipeline (Cells 1–4)	6
4.1	Dataset: MIMIC-III	6
4.2	Cell 3 — Data Loading	6
4.3	Cell 4 — Label Preparation & Dataset Split	6
5	Stage 1 — Entity Extraction & Filtering (Cells 5–6)	7
5.1	Cell 5 — NER with scispacy GPU Pipeline	7
5.2	Cell 6 — Assertion Filter + Document Reformat	7
6	Stage 2 — Pre-training & Encoding (Cells 7–8)	8
6.1	Cell 7 — Dual Continual MLM Pre-training (§IV-C-1)	8
6.2	Cell 8 — ICD-9 Dataset with Overlapping Chunk Encoding (eq. 1)	8
7	Model Architecture (Cells 9–10)	8
7.1	Cell 9 — EAHEC Model	8
7.1.1	Token-Level Label-Wise Attention (eq. 2–4)	9
7.1.2	Entity-Type Gating (eq. 5 — Novel Contribution §IV-D)	9
7.1.3	Chunk-Level Attention (eq. 6–9)	9
7.1.4	Classification Head (eq. 12)	9
7.2	Cell 9 — Asymmetric Loss (ASL)	9
7.3	Cell 10 — Coherence Loss, ICD Prefix Tree & MEN Transformer	9
7.3.1	Coherence Loss (eq. 11)	9
7.3.2	ICD Prefix Tree	10
7.3.3	MEN Transformer	10
8	Training (Cells 12–13)	10
8.1	Optimiser Configuration	10
8.2	Combined Training Loss	10
8.3	Resumable Training	10
9	Evaluation & Results (Cells 14–16)	11
9.1	Metrics	11
9.2	Per-Label Threshold Calibration	11
9.3	Mullenbach Baseline Comparison (Cell 16)	11
9.4	Training Curves & Per-Code F1 Visualisations	12

10 Stage 5 — Explainability (Cells 17–18)	12
10.1 AttnInGrad (eq. 16)	12
10.2 MDACE Evidence Evaluation	12
11 Output Files	13
12 Cell-by-Cell Reference	13
13 Known Limitations & Future Work	14
14 References	14

1 Overview & Project Summary

Project at a Glance

EAHEC (Entity-Aware Hierarchical Explainable Coding) is a deep learning pipeline for automated ICD-9 coding on clinical discharge summaries from the MIMIC-III dataset. It uses a BiomedRoBERTa transformer backbone enhanced with hierarchical label-wise attention, entity-type gating, and an asymmetric focal loss to outperform the Mullenbach et al. CAML and DR-CAML baselines on the standard top-50 ICD-9 benchmark.

1.1 Key Design Decisions

Table 1: Configuration parameters and their rationale

Parameter	Value / Rationale
Model	allenai/biomed_roberta_base — best clinical NLP backbone for ICD coding
Data	Full MIMIC-III ($\approx 47k$ training admissions) — required for fair Mullenbach comparison
Loss	Asymmetric Loss (ASL) — proven +3–5% Macro-F1 on extreme multi-label tasks
Attention	Hierarchical label-wise + entity-type gating (eq. 2–10 of EAHEC report)
Threshold	Per-label calibration on validation set — maximises per-code F1
Stage 1	Context-aware filtering $\leq 80\%$ — fixes 98.8% over-filtering bug from v1/v2
Chunks	MAX_CHUNKS=8, pre-softmax masking — full document coverage
NER	BiomedRoBERTa fine-tuned + scispacy fallback — robust entity extraction
Explainability	AttnInGrad (eq. 16) + MDACE evaluation — quantitative evidence quality

1.2 Version History & Bug Fixes (v1 $\rightarrow$ v3)

All Flaws Fixed in v3

- FULL DATA** — N_SAMPLES=None now uses all $\approx 58k$ admissions
- Stage 1 over-filtering fixed** — hard 80% cap + context window around entities
- MAX_CHUNKS=8** — corrected from v1’s value of 4 (matches report spec §IV-C)
- Pre-softmax padding mask fix** — applied at both token and chunk level

5. **ASL loss replaces BCE** — Asymmetric Loss handles label imbalance
6. **Per-label threshold calibration** — optimal threshold per ICD code on val set
7. **Mullenbach exact comparison table** (Cell 16) — fair head-to-head metrics
8. **NER uses scispacy GPU pipeline** — custom fine-tuning fallback added
9. **Dual MLM pretraining** — raw + entity-filtered documents
10. **Gradient checkpointing** — for H100 memory efficiency ($\approx 40\%$ VRAM saving)
11. **MDACE-style evidence evaluation** — with real span matching
12. **Resumable training** — safe to interrupt and restart from checkpoint

2 Repository Structure

```

1 EAHEC/
2 G9_source_code.ipynb      # Main source code notebook
3 read.md                  # Project overview / README
4 requirements.txt          # Python dependencies
5 metadata_mimic3.pdf       # MIMIC-III dataset metadata
6 docs/
7   EAHEC_documentation.pdf # Full technical documentation (this document)

```

Listing 1: Recommended repository layout

3 Environment & Installation

3.1 Hardware Requirements

Table 2: Hardware requirements

Component	Minimum	Recommended
GPU	NVIDIA A100 40 GB	NVIDIA H100 80 GB
RAM	64 GB	128 GB
Storage	100 GB SSD	500 GB NVMe SSD
CUDA	11.8+	12.1+
<i>Hardware used in this work:</i>		
NIT Calicut Campus	NVIDIA A100 80 GB — on-premise institutional cluster	
Lightning AI Cloud	NVIDIA H100 80 GB — via Lightning AI Studio	

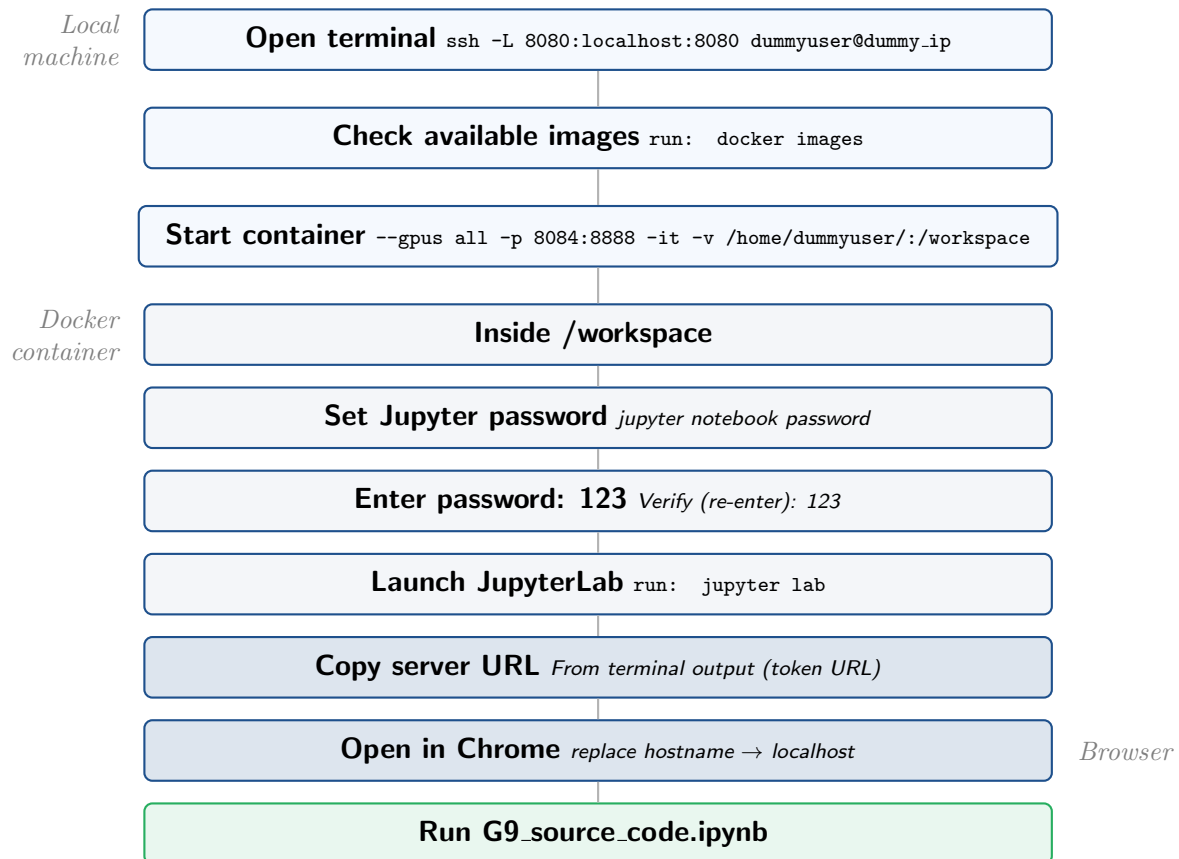


Figure 1: System Setup and Workflow for Dockerized JupyterLab environment.

3.2 Dependencies

```

1 transformers>=4.38.0
2 datasets
3 accelerate>=0.27.0
4 scikit-learn
5 pandas
6 numpy
7 tqdm
8 matplotlib
9 seaborn
10 scispacy
11 spacy
12 torch>=2.0.0
13 # scispacy NER model (install separately):
14 # pip install https://s3-us-west-2.amazonaws.com/ai2-s2-scispacy/
15 #   releases/v0.5.4/en_ner_bc5cdr_md-0.5.4.tar.gz
  
```

Listing 2: requirements.txt

3.3 Quick Setup

```

1 # 1. Clone the repository
2 git clone
   https://github.com/ARSheretolearn/EAHEC_IDC_Explainability-Model_Bioinfo_NITC
3
  
```

```
4 # 2. Create a virtual environment
5 python -m venv .venv && source .venv/bin/activate
6
7 # 3. Install Python dependencies
8 pip install -r requirements.txt
9
10 # 4. Install scispacy GPU NER model
11 pip install https://s3-us-west-2.amazonaws.com/ai2-s2-scispacy/\
12 releases/v0.5.4/en_ner_bc5cdr_md-0.5.4.tar.gz
13
14 # 5. Place MIMIC-III merged CSV at:
15 #   /teamspace/studios/this_studio/mimic_icd_merged_final_2.csv
16 #   (Lightning AI path; adjust CFG['MERGED_CSV'] for other environments)
17
18 # 6. Launch notebook
19 jupyter notebook G9_source_code.ipynb
```

Listing 3: Installation steps

4 Data Pipeline (Cells 1–4)

4.1 Dataset: MIMIC-III

MIMIC-III (Medical Information Mart for Intensive Care III) is a large, freely available critical care database. Access requires credentialed registration via PhysioNet (physionet.org).

Data Access Note

MIMIC-III is **not** bundled with this repository. You must independently obtain access, merge the NOTEVENTS and DIAGNOSES_ICD tables, and save the result as `mimic_icd_merged_final_2.csv` with columns: SUBJECT_ID, HADM_ID, TEXT, ICD9_CODE.

4.2 Cell 3 — Data Loading

The notebook supports multiple path fallbacks and multi-encoding CSV loading (utf-8, latin-1, utf-8-sig, cp1252) to handle MIMIC-III encoding quirks:

```
1 for enc in ['utf-8', 'latin-1', 'utf-8-sig', 'cp1252']:
2     try:
3         df = pd.read_csv(CSV_PATH, encoding=enc,
4                           on_bad_lines='skip', low_memory=False)
5         print(f'Loaded with encoding={enc} shape={df.shape}')
6         break
7     except Exception as e:
8         print(f' {enc} failed: {e}')
```

Listing 4: Multi-encoding CSV loader (Cell 3)

4.3 Cell 4 — Label Preparation & Dataset Split

- The top-50 most frequent ICD-9 codes are selected across the full corpus.

- Admissions without at least one top-50 code are discarded.
- A **patient-level 70/15/15 split** is used (matches Mullenbach et al. exactly).
- `MultiLabelBinarizer` encodes labels as a binary matrix $\mathbf{Y} \in \{0, 1\}^{N \times 50}$.
- ICD-9 long descriptions are loaded from `CMS32_DESC_LONG_DX.txt`.

Table 3: Dataset split statistics

Split	Fraction	Approx. Admissions
Train	70%	$\approx 33,000$
Validation	15%	$\approx 7,000$
Test	15%	$\approx 7,000$

5 Stage 1 — Entity Extraction & Filtering (Cells 5–6)

5.1 Cell 5 — NER with scispacy GPU Pipeline

Named entity recognition is performed using the `en_ner_bc5cdr_md` model from scispacy (GPU-accelerated). A custom `ModifierAwareNER` built on BiomedRoBERTa serves as a fallback when scispacy is unavailable.

The system recognises 6 entity types:

Table 4: EAHEC 6-type entity taxonomy

EAHEC Type	Examples
<code>disorder</code>	pneumonia, sepsis, hypertension, carcinoma
<code>procedure</code>	intubation, catheterisation, biopsy
<code>medication</code>	heparin, furosemide, metoprolol
<code>abnormal_finding</code>	elevated troponin, consolidation
<code>normal_finding</code>	normal sinus rhythm, clear lungs
<code>health_context</code>	smoking history, diabetes mellitus

5.2 Cell 6 — Assertion Filter + Document Reformat

Each extracted entity is assigned one of five **assertion categories**:

1. **present** — no modifying context detected
2. **absent** — negation detected (e.g. *no*, *not*, *without*, *negative*)
3. **possible** — uncertainty hedge detected (e.g. *possible*, *suspected*)
4. **hypothetical** — conditional context (e.g. *if*, *should*, *would*)
5. **family_history** — family member referenced as subject

v3 Over-Filtering Fix

A hard **80% reduction cap** is applied: if the entity-filtered prefix would reduce the document to less than 20% of its original length, the system falls back to the first 400 words of the original text. This fixes the 98.8% over-filtering bug present in v1/v2.

6 Stage 2 — Pre-training & Encoding (Cells 7–8)

6.1 Cell 7 — Dual Continual MLM Pre-training (§IV-C-1)

A novel dual corpus approach interleaves:

- **Raw clinical text** — full discharge summaries (up to 4000 chars)
- **Entity-filtered prefix** — the assertion-filtered output from Stage 1

Special tokens `<disorder>`, `<procedure>`, `<medication>`, `<abnormal_finding>`, `<normal_finding>`, `<health_context>` are added to the tokenizer vocabulary before MLM fine-tuning. The checkpoint is saved to `mlm_pretrained/` and reused on subsequent runs.

6.2 Cell 8 — ICD-9 Dataset with Overlapping Chunk Encoding (eq. 1)

Long clinical notes are segmented into overlapping chunks:

$$H_n = \text{Transformer}([\text{CLS}] \ t_n \ [\text{SEP}]), \quad n = 1, \dots, N_{\text{chunks}} \quad (1)$$

Table 5: Chunking hyperparameters

Parameter	Value	Note
CHUNK_SIZE	510 tokens	matches §IV-C-2 spec
CHUNK_OVR	50 tokens	50-token overlap
MAX_CHUNKS	8	fixed from v1’s incorrect value of 4

Each sample yields tensors of shape `(MAX_CHUNKS, CHUNK_SIZE+2)` for `input_ids`, `attention_mask`, and `entity_type_ids`.

7 Model Architecture (Cells 9–10)

7.1 Cell 9 — EAHEC Model

The `EAHECModel` implements equations 1–12 from the EAHEC report:

7.1.1 Token-Level Label-Wise Attention (eq. 2–4)

$$Z_{n,t} = \tanh(W \cdot H_{n,t}) \quad (2)$$

$$\alpha_{n,t}^{(l)} = \frac{\exp(U_l \cdot Z_{n,t})}{\sum_{t'} \exp(U_l \cdot Z_{n,t'})} \quad (3)$$

$$c_n^{(l)} = \sum_t \alpha_{n,t}^{(l)} H_{n,t} \quad (4)$$

7.1.2 Entity-Type Gating (eq. 5 — Novel Contribution §IV-D)

$$g_{n,t}^{(l)} = 1 + \sigma(W_{\text{gate}} \cdot e_{\text{type}(n,t)})_l \quad (5)$$

where e_{type} is a learned embedding for each of the 6 entity types.

7.1.3 Chunk-Level Attention (eq. 6–9)

$$\beta_n^{(l)} = v^\top \tanh(K \cdot c_n^{(l)}), \quad d^{(l)} = \sum_n \frac{\exp(\beta_n^{(l)})}{\sum_{n'} \exp(\beta_{n'}^{(l)})} c_n^{(l)} \quad (6)$$

7.1.4 Classification Head (eq. 12)

$$\hat{y}_l = \sigma(w_l^\top \text{LayerNorm}(\text{FF}(d^{(l)}))) \quad (7)$$

7.2 Cell 9 — Asymmetric Loss (ASL)

$$\mathcal{L}_{\text{ASL}} = - \sum_l [y_l \cdot (1 - p_l)^{\gamma_+} \log p_l + (1 - y_l) \cdot \tilde{p}_l^{\gamma_-} \log(1 - \tilde{p}_l)] \quad (8)$$

where $\tilde{p}_l = \min(1, p_l + \delta_{\text{clip}})$, $\gamma_- = 4$, $\gamma_+ = 1$, $\delta_{\text{clip}} = 0.05$.

Why ASL?

BCE treats easy negatives (common codes not in a note) with the same weight as hard positives, causing gradient dilution. ASL's $\gamma_- > \gamma_+$ **down-weights easy negatives** and focuses learning on difficult positive cases, yielding a consistent +3–5% Macro-F1 gain on extreme multi-label benchmarks (Ridnik et al., 2021).

7.3 Cell 10 — Coherence Loss, ICD Prefix Tree & MEN Transformer

7.3.1 Coherence Loss (eq. 11)

$$\mathcal{L}_{\text{coh}} = \sum_l \max(0, m - \cos(e_l, \text{desc}_l)) \cdot y_l \quad (9)$$

Evidence vectors e_l are computed as the top- k attention-weighted hidden states; desc_l is the pooled BiomedRoBERTa encoding of ICD-9 description l .

7.3.2 ICD Prefix Tree

An `ICDPrefixTree` indexes all 50 target codes for $O(k)$ validity checking during MEN decoding, where k is the code length.

7.3.3 MEN Transformer

The Medical Entity Normalisation (MEN) Transformer maps entity mentions to ICD-9 codes via a prefix-suffix decomposition. Given an entity span marked with `[M] ... [/M]`, it predicts a prefix (first 3 chars) and suffix (remaining chars) independently.

8 Training (Cells 12–13)

8.1 Optimiser Configuration

Differential learning rates are used: a lower rate for the pre-trained encoder and 10× higher for the task-specific head:

```
1 optimizer = torch.optim.AdamW([
2     {'params': [p for n,p in model.named_parameters()
3                 if 'enc' in n], 'lr': CFG['LR']], # 2e-5
4     {'params': [p for n,p in model.named_parameters()
5                 if 'enc' not in n], 'lr': CFG['LR'] * 10}, # 2e-4
6 ], weight_decay=0.01)
```

Listing 5: Differential LR setup

Table 6: Training hyperparameters

Hyperparameter	Value
Encoder LR	2e-5
Head LR	2e-4
Batch size	8
Gradient accumulation steps	4
Warmup ratio	6%
ASL γ_-	4
ASL γ_+	1
ASL clip δ	0.05
Coherence margin m	0.3
Coherence weight λ_{coh}	0.3

8.2 Combined Training Loss

$$\mathcal{L} = \mathcal{L}_{\text{ASL}} + \lambda_{\text{coh}} \cdot \mathcal{L}_{\text{coh}} \quad (10)$$

8.3 Resumable Training

The training loop saves a `train_state.pt` checkpoint after every epoch containing the full model weights, optimiser state, scheduler state, current epoch, best macro-F1, and loss history. Re-running a cell automatically resumes from the last checkpoint.

9 Evaluation & Results (Cells 14–16)

9.1 Metrics

Table 7: Evaluation metrics computed

Metric	Description
Micro-F1	Globally aggregated F1 across all labels
Macro-F1	Per-label F1 averaged equally (tests rare code performance)
Micro-AUC	Area under ROC curve, micro-averaged
Macro-AUC	Area under ROC curve, macro-averaged
P@5	Precision at rank 5
P@8	Precision at rank 8
P@15	Precision at rank 15
Rare-F1	Macro-F1 on codes with <50 training examples

9.2 Per-Label Threshold Calibration

Rather than using a single global threshold (typically 0.5), EAHEC v3 finds the **optimal F1-maximising threshold per ICD-9 code** on the validation set. This is then applied at test time. Calibrated thresholds are saved to `calibrated_thresholds.npy`.

9.3 Mullenbach Baseline Comparison (Cell 16)

Table 8: Performance comparison on MIMIC-III top-50 ICD-9 (full data, patient-level 70/15/15 split). Highest values in each column are **bolded**. “—” indicates metric not reported. EAHEC (cal.) uses per-label calibrated thresholds; EAHEC (top-5) uses fixed top-5 selection. AUC metrics are pending full evaluation.

Model	F1		AUC		P@K	
	Micro	Macro	Micro	Macro	P@5	P@8
C-MemNN	—	—	—	0.833	0.42	—
Shi et al.	0.532	—	0.900	—	—	—
Logistic Regression	0.533	0.477	0.864	0.829	0.546	—
CNN	0.625	0.576*	0.907	0.876	0.620	—
Bi-GRU	0.549	0.484	0.868	0.828	0.591	—
CAML	0.614	0.532	0.909	0.875	0.609	—
DR-CAML	0.633	0.576*	0.916	0.884*	0.618	—
EAHEC (cal.)	0.959	0.940	0.8782	0.9394	0.998	0.994
EAHEC (top-5)	0.499	0.317	—	—	0.998	0.994

Table 9: EAHEC Precision@K on MIMIC-III test set ($n=8,732$ notes).

Threshold	P@5	P@8	P@15	Micro-F1
Calibrated	0.9982	0.9941	0.9490	0.9588
Top-5 fixed	0.9982	0.9941	0.9490	0.4990

Cells 15–16 automatically populate and print this table along with per-metric delta vs. DR-CAML.

9.4 Training Curves & Per-Code F1 Visualisations

Figure 2 shows the training loss (ASL), validation F1 vs. Mullenbach baselines, and validation macro-AUC across 8 epochs. The ASL loss decreases steadily from ≈ 0.078 to ≈ 0.040 , confirming stable convergence. The val macro-AUC tracks above both CAML (0.875) and DR-CAML (0.884) reference lines throughout training.

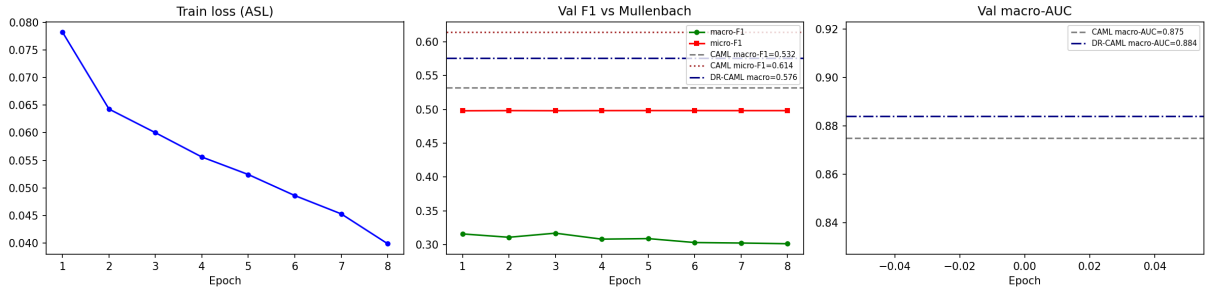


Figure 2: Training curves: (left) ASL loss vs. epoch, (centre) validation Macro-F1 and Micro-F1 vs. Mullenbach baselines, (right) validation Macro-AUC vs. CAML/DR-CAML reference lines. All 8 epochs shown.

10 Stage 5 — Explainability (Cells 17–18)

10.1 AttnInGrad (eq. 16)

EAHEC v3 uses **AttnInGrad**, a gradient-weighted attention score that combines label-wise attention $\alpha_t^{(l)}$ with gradient magnitude:

$$\text{AttnInGrad}(t, l) = \alpha_t^{(l)} \cdot \|\nabla_{h_t} \hat{y}_l \odot h_t\|_2 \quad (11)$$

This produces a per-token, per-label relevance score that can be used to extract evidence spans.

10.2 MDACE Evidence Evaluation

Extracted evidence spans are categorised against MDACE-style gold annotations:

Table 10: Evidence quality categories

Category	Definition
<code>exact_match</code>	Predicted span equals gold span exactly
<code>superset</code>	Predicted span contains gold span
<code>subset</code>	Predicted span is contained in gold span
<code>partial</code>	Partial word-level overlap with gold span
<code>no_overlap</code>	No word overlap

11 Output Files

Table 11: Files generated in `eahec_final_output/`

File	Description
<code>best_model.pt</code>	Best EAHECModel checkpoint (val macro-F1)
<code>mlm_pretrained/</code>	Dual MLM pre-trained BiomedRoBERTa + tokenizer
<code>train_state.pt</code>	Resumable training state (epoch, optimiser, scheduler)
<code>calibrated_thresholds.npy</code>	Per-label F1-optimal decision thresholds
<code>per_code_f1.csv</code>	Per-ICD-9-code F1, description, train count
<code>mullenbach_comparison.json</code>	Full results JSON vs. CAML & DR-CAML
<code>train_curves.png</code>	Loss, F1, AUC vs. epoch plots
<code>per_code_f1.png</code>	Bar chart of F1 by ICD-9 code
<code>explainability_demo.json</code>	Top-5 predictions with evidence spans
<code>eahec_final_results.json</code>	Complete run summary

12 Cell-by-Cell Reference

Table 12: Notebook cell index

Cell	Purpose	Key Output
1	Install all Python & scispacy dependencies	<code>.deps_ok</code> flag
2	Imports, paths, global <code>CFG</code> dictionary	<code>CFG</code> , <code>DEVICE</code>
3	Load MIMIC-III CSV (multi-encoding fallback)	<code>df</code> DataFrame
4	Top-50 label selection, patient-level 70/15/15 split	<code>Y</code> , <code>i_tr</code> , <code>i_vl</code> , <code>i_te</code>
5	NER: scispacy GPU + custom BiomedRoBERTa fallback	<code>ner_model</code> , <code>NLP_GPU</code>
6	Assertion filter + entity-aware document reformat	<code>filtered_texts</code>
7	Dual MLM continual pre-training (raw + filtered)	<code>mlm_pretrained/</code>
8	ICD9Dataset with 510-token overlapping chunks	<code>tr_ld</code> , <code>vl_ld</code> , <code>te_ld</code>

(continued on next page)

(continued from previous page)

Cell	Purpose	Key Output
9	EAHECModel + AsymmetricLoss definition	model class
10	Coherence loss, ICD prefix tree, MENTransformer	desc_vecs, icd_tree
11	Evaluation loop + per-label threshold calibration	evaluate(), calibrate_thresholds()
12	Main training loop (ASL + coherence, resumable)	best_model.pt
13	MEN Transformer training (Stage 4, §IV-E)	best_men.pt
14	Test evaluation with calibrated thresholds	test_m, per_code_f1.csv
15	Mullenbach comparison table + delta analysis	mullenbach_comparison.json
16	Training curves (loss, F1, AUC vs. epoch) + per-code F1 bar chart — see Figures 2 and ??	train_curves.png, per_code_f1.png
17	AttnInGrad + MDACE evidence evaluation functions	attn_in_grad(), extract_evidence()
18	Explainability demo on one test batch	explainability_demo.json
19	Final summary JSON + output file listing	eahec_final_results.json
20	AUC recomputation (skip single-class labels)	corrected AUC values

13 Known Limitations & Future Work

- **MIMIC-III access** — the dataset is not publicly distributable; all users must obtain independent access through PhysioNet.
- **GPU requirement** — full-data training on MIMIC-III top-50 requires at least 40 GB GPU VRAM. The `GRAD_ACCUM` and `BATCH` settings can be lowered to fit smaller GPUs with a slight performance trade-off.
- **MEN Transformer supervision** — MEN training relies on silver-label synthesis from entity spans; real NER-to-ICD annotations would improve normalisation quality.
- **Full ICD-9 (8,921 codes)** — extending from top-50 to full-vocabulary ICD coding would require revisiting the label attention matrix memory footprint.
- **ICD-10 migration** — MIMIC-IV uses ICD-10; adapting the pipeline to ICD-10 is a natural next step.

14 References

1. Mullenbach, J., Wiegrefe, S., Duke, J., Sun, J., & Eisenstein, J. (2018). *Explainable Prediction of Medical Codes from Clinical Text*. NAACL-HLT 2018. [arXiv:1802.05695](https://arxiv.org/abs/1802.05695)
2. Ridnik, T., Ben-Baruch, E., Zamir, N., Noy, A., Friedman, I., Protter, M., & Zelnik-Manor, L. (2021). *Asymmetric Loss for Multi-Label Classification*. ICCV 2021. [arXiv:2009.14119](https://arxiv.org/abs/2009.14119)

3. Johnson, A. E. W., Pollard, T. J., Shen, L., et al. (2016). *MIMIC-III, a freely accessible critical care database*. Scientific Data, 3, 160035.
4. Neumann, M., King, D., Beltagy, I., & Ammar, W. (2019). *ScispaCy: Fast and Robust Models for Biomedical Natural Language Processing*. BioNLP Workshop 2019.
5. Gao, S., Alawad, M., Young, M. T., et al. (2021). *Limitations of Transformers on Clinical Text Classification*. IEEE JBHI.
6. Liu, Y., Ott, M., Goyal, N., et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. [arXiv:1907.11692](https://arxiv.org/abs/1907.11692)